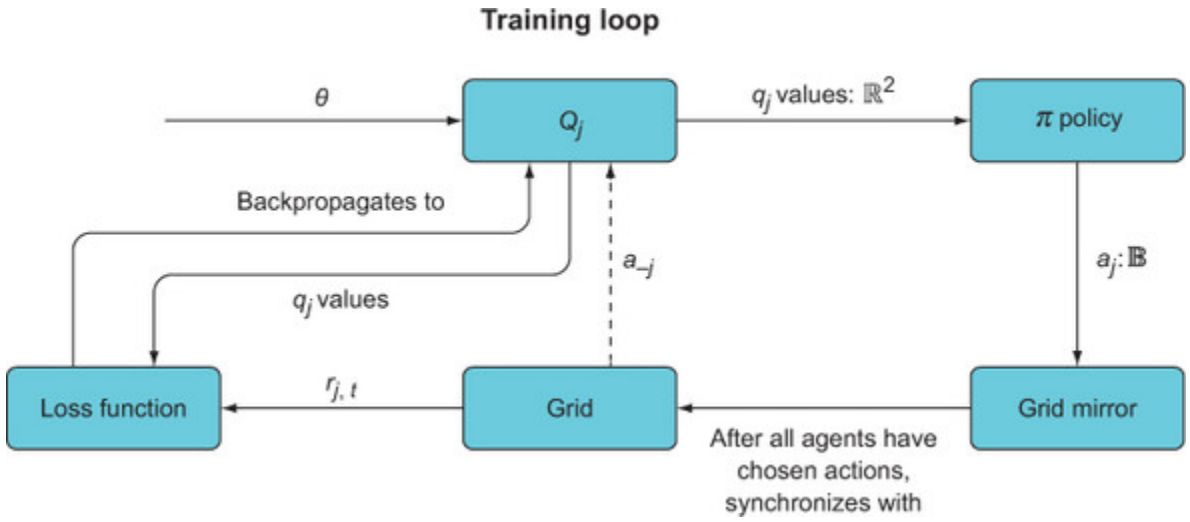
we thought it was useful to show how straightforward it is to model each agent separately. In other environments, where agents may have differing optimal policies, you would need to use separate DQNs for each one.

We've simplified this main training function a bit to avoid distractions (figure 9.16). First, notice that the policy we use is a greedy policy. The agent takes the action that has the highest Q value every time; there's no epsilon-greedy policy where we sometimes take a random action. In general, some sort of exploration strategy is necessary, but this is such a simple problem that it still works. In the next section, we will solve a 2D Ising model on a square grid, and in that case we will use a softmax policy where the temperature parameter will model the actual physical temperature of the system of electrons we are trying to model.

Figure 9.16. This is a string diagram for the main training loop. For each agent, j, the corresponding Q function accepts a parameter vector and the joint-action vector for agent j, denoted a–j. The Q function outputs a 2-element Q value vector that is input to the policy function, and it chooses an action (a binary digit) that then gets stored in a mirror (clone) of the grid environment. After all agents have chosen actions, the mirrored grid synchronizes with the main grid. The rewards are generated for each agent and are passed to the loss function, which computes a loss and backpropagates the loss into the Q function, and ultimately into the parameter vector for updating.



The other simplification we made is that the target Q value is set to be r_{t+1} (the reward after taking the action). Normally it would be $r_{t+1} + \gamma * V(S_{t+1})$, where the last term is the discount factor gamma times the value of the state